

# ConceptNet Analysis - Query List

## Contents

|                                                    |          |
|----------------------------------------------------|----------|
| <b>1 FAMILY A (Local Retrieval)</b>                | <b>2</b> |
| 1.1 Single Node Search by URI . . . . .            | 2        |
| 1.2 Full Relation Table Search . . . . .           | 2        |
| 1.3 Relation 1-hop Search . . . . .                | 2        |
| 1.4 Directed vs. Undirected 1-hop Search . . . . . | 3        |
| 1.5 Relation Pair Intersection . . . . .           | 4        |
| 1.6 Range Selection . . . . .                      | 4        |
| 1.7 Neighborhoods Set Difference . . . . .         | 5        |
| <b>2 FAMILY B (Traversal Paths)</b>                | <b>5</b> |
| 2.1 K-Hop Neighborhood . . . . .                   | 5        |
| 2.2 Transitive Closure . . . . .                   | 6        |
| 2.3 Concepts Pair Shortest Chain . . . . .         | 6        |
| 2.4 Weighted Path Scoring . . . . .                | 7        |
| 2.5 Anchored Triangle . . . . .                    | 7        |
| 2.6 Neighborhood Degree Profile . . . . .          | 8        |
| <b>3 FAMILY C (Global Retrieval)</b>               | <b>9</b> |
| 3.1 Concept Hubs Ranking . . . . .                 | 9        |
| 3.2 Relation-Class Type Matrix . . . . .           | 9        |
| <b>4 FAMILY D (Schema and Integrity Findings)</b>  | <b>9</b> |
| 4.1 Violation Check . . . . .                      | 9        |
| 4.2 Invalid Insert . . . . .                       | 10       |
| 4.3 Concept Delete . . . . .                       | 11       |

# 1 FAMILY A (Local Retrieval)

## 1.1 Single Node Search by URI

**Task:** Find a concept using a given URI.

### MySQL

```
SELECT uri, name, node_type, label_source
FROM nodes WHERE uri = '/c/en/photosynthesis/n';
```

### Memgraph (Cypher)

```
MATCH (n:Concept {uri: '/c/en/photosynthesis/n'})
RETURN n.uri AS uri, n.name AS name,
CASE WHEN n:EntityNode THEN 'EntityNode'
      WHEN n:ActionEventNode THEN 'ActionEventNode'
      ELSE 'PropertyNode' END AS node_type,
n.label_source AS label_source;
```

## 1.2 Full Relation Table Search

**Task:** Return all outgoing edges of a given concept, as subject–relation–object triples ranked by weight.

### MySQL

```
SELECT s.name AS subject, r.relation_name AS relation,
       o.name AS object, e.weight
FROM nodes s
JOIN edges e ON e.subject_id = s.node_id
JOIN nodes o ON o.node_id = e.object_id
JOIN relations r ON r.relation_id = e.relation_id
WHERE s.name = 'photosynthesis'
ORDER BY e.weight DESC LIMIT 25;
```

### Memgraph (Cypher)

```
MATCH (n:Concept {name: 'photosynthesis'})-[r]->(m:Concept)
RETURN n.name AS subject, type(r) AS relation, m.name AS object, r.
weight
ORDER BY r.weight DESC LIMIT 25;
```

## 1.3 Relation 1-hop Search

**Task:** Given a concept and a relation, return the concepts connected to it by that relation — where the query constrains the endpoint type (e.g., only ActionEvent nodes for CapableOf).

## MySQL

```
SELECT o.name AS action
FROM nodes s
JOIN edges e ON e.subject_id = s.node_id
JOIN nodes o ON o.node_id = e.object_id
WHERE s.name = 'animals' AND e.relation_id = 3;
```

## Memgraph (Cypher)

```
MATCH (:Concept {name: 'animals'})-[:CapableOf]->(a:ActionEventNode)
RETURN a.name;
```

### 1.4 Directed vs. Undirected 1-hop Search

**Task:** Search 1-hop neighbors comparing reverse direction and undirected search.

## MySQL

```
-- (a) reverse direction
SELECT s.name AS thing
FROM nodes o
JOIN edges e ON e.object_id = o.node_id
JOIN nodes s ON s.node_id = e.subject_id
WHERE o.name = 'desert' AND e.relation_id = 2,
ORDER BY thing;

-- (b) undirected
SELECT CASE WHEN e.subject_id = n.node_id THEN o.name ELSE s.name END AS
neighbor
FROM nodes n
JOIN edges e ON (e.subject_id = n.node_id OR e.object_id = n.node_id)
JOIN nodes s ON s.node_id = e.subject_id
JOIN nodes o ON o.node_id = e.object_id
WHERE n.name = 'person' AND e.relation_id = 1,
ORDER BY neighbor;
```

## Memgraph (Cypher)

```
// (a) reverse direction
MATCH (thing:Concept)-[:AtLocation]->(:Concept {name: 'laboratory'})
RETURN thing.name AS thing
ORDER BY thing;

// (b) undirected
MATCH (:Concept {name: 'person'})-[:Antonym]-(y:Concept)
RETURN y.name AS neighbor
ORDER BY neighbor;
```

## 1.5 Relation Pair Intersection

**Task:** Given two concepts and a relation, find the concepts that both are connected to via that relation.

### MySQL

```
SELECT loc.name AS shared
FROM nodes a
JOIN edges e1 ON e1.subject_id = a.node_id AND e1.relation_id = 2
JOIN edges e2 ON e2.object_id = e1.object_id AND e2.relation_id = 2
JOIN nodes b ON b.node_id = e2.subject_id AND b.name = 'beaver'
JOIN nodes loc ON loc.node_id = e1.object_id
WHERE a.name = 'horse',
ORDER BY shared;
```

### Memgraph (Cypher)

```
MATCH (a:Concept {name: 'horse'})-[:IsA]->(loc:Concept)
      <-[:IsA]-(b:Concept {name: 'beaver'})
RETURN loc.name AS shared
ORDER BY shared;
```

## 1.6 Range Selection

**Task:** Given two concept names, return all concepts whose name falls alphabetically between them (a range selection on the name index).

### MySQL

```
SELECT n.name, n.node_type
FROM nodes
WHERE n.name BETWEEN 'cell' AND 'gene'
ORDER BY n.name;
```

### Memgraph (Cypher)

```
MATCH (n:Concept)
WHERE n.name >= 'cell' AND n.name <= 'gene'
RETURN n.name,
       CASE WHEN n:EntityNode THEN 'EntityNode'
            WHEN n:ActionEventNode THEN 'ActionEventNode'
            ELSE 'PropertyNode' END
       AS node_type
ORDER BY n.name;
```

## 1.7 Neighborhoods Set Difference

**Task:** Given two concepts and a relation, find the concepts connected to the first via that relation but not to the second.

### MySQL

```
SELECT DISTINCT s.name FROM edges e
JOIN nodes s ON s.node_id = e.subject_id
JOIN nodes a ON a.node_id = e.object_id
WHERE a.name = 'human' AND e.relation_id = 2
AND NOT EXISTS (SELECT 1 FROM edges e2
                JOIN nodes b ON b.node_id = e2.object_id
                WHERE e2.subject_id = e.subject_id
                  AND e2.relation_id = 2
                  AND b.name = 'body'),
ORDER BY name;
```

### Memgraph (Cypher)

```
MATCH (x:Concept)-[:AtLocation]->(:Concept {name: 'human'})
WHERE NOT (x)-[:AtLocation]->(:Concept {name: 'body'})
RETURN DISTINCT x.name AS name
ORDER BY name;
```

## 2 FAMILY B (Traversal Paths)

### 2.1 K-Hop Neighborhood

**Task:** Given a number K and a concept, find the set of concepts reachable in K weight-filtered hops starting from the concept.

### MySQL

```
SELECT DISTINCT x.name -- k = 2 shown; one JOIN per level
FROM nodes n
JOIN edges e1 ON e1.subject_id = n.node_id
JOIN edges e2 ON e2.subject_id = e1.object_id
JOIN nodes x ON x.node_id = e2.object_id
WHERE n.name = 'enzyme' AS name
      AND e1.weight >= 0.5 AND e2.weight >= 0.5
ORDER BY name LIMIT 100;
```

### Memgraph (Cypher)

```
MATCH (n:Concept {name: 'enzyme'})-[r1]->()-[r2]->(x:Concept)
WHERE r1.weight >= 0.5 AND r2.weight >= 0.5
RETURN DISTINCT x.name AS name
ORDER BY name LIMIT 100;
```

## 2.2 Transitive Closure

**Task:** Given a concept, return all its transitive superclasses reachable through chains of IsA edges, up to depth 5.

### MySQL

```
WITH RECURSIVE anc(node_id, depth) AS (
  SELECT node_id, 0 FROM nodes WHERE name = 'mitochondrion'
  UNION ALL
  -- UNION DISTINCT not allowed in
  -- the recursive member
  SELECT e.object_id, a.depth + 1
  FROM anc a JOIN edges e ON e.subject_id = a.node_id
  WHERE e.relation_id = 23 AND a.depth < 5
)
SELECT DISTINCT n.name AS superclass
FROM anc a JOIN nodes n ON n.node_id = a.node_id
WHERE a.depth > 0,
ORDER BY superclass;
```

### Memgraph (Cypher)

```
MATCH (:Concept {name: 'mitochondrion'})-[:IsA*1..5]->(sup:Concept)
RETURN DISTINCT sup.name AS superclass
ORDER BY superclass;
```

## 2.3 Concepts Pair Shortest Chain

**Task:** Given two concepts, find the shortest path that connects the two concepts using any relation within a bounded number of hops.

## MySQL

```
WITH RECURSIVE walk(node_id, depth, path) AS (  
  SELECT node_id, 0, CAST(node_id AS CHAR(2000))  
  FROM nodes WHERE name = 'photosynthesis'  
  UNION ALL  
  SELECT e.other_id, w.depth + 1, CONCAT(w.path, ',', e.other_id)  
  FROM walk w  
  JOIN ( SELECT subject_id AS from_id, object_id AS other_id FROM  
    edges  
    UNION ALL  
    SELECT object_id AS from_id, subject_id AS other_id FROM  
    edges ) e  
  ON e.from_id = w.node_id  
  WHERE w.depth < 3 AND FIND_IN_SET(e.other_id, w.path) = 0  
)  
SELECT depth AS hops FROM walk  
WHERE node_id IN (SELECT node_id FROM nodes WHERE name = 'human')  
ORDER BY depth, path LIMIT 1;
```

## Memgraph (Cypher)

```
MATCH p = (a:Concept {name: 'photosynthesis'})-[r*BFS..3]-(b:Concept {  
  name: 'human'})  
RETURN length(p) AS hops  
ORDER BY hops LIMIT 1;
```

## 2.4 Weighted Path Scoring

**Task:** Given two concepts, rank all paths between two concepts by the product of their edge weights, within a bounded number of hops.

## MySQL

```
-- No existing counterpart. SQL cannot enumerate paths before  
aggregating them.
```

## Memgraph (Cypher)

```
MATCH p = (a:Concept {name: 'photosynthesis'})-[r*..3]->(b:Concept {name  
  : 'plant'})  
RETURN p, reduce(acc = 1.0, rel IN relationships(p) | acc * rel.weight)  
AS score  
ORDER BY score DESC LIMIT 5;
```

## 2.5 Anchored Triangle

**Task:** Given a concept, find a 3-hop path that connects the concept to itself through any relation.

## MySQL

```
SELECT DISTINCT y.uri AS hop1_uri, y.name AS hop1, z.uri AS hop2_uri, z.
    name AS hop2
FROM nodes n
JOIN edges e1 ON e1.subject_id = n.node_id
JOIN edges e2 ON e2.subject_id = e1.object_id
JOIN edges e3 ON e3.subject_id = e2.object_id
WHERE n.name = 'blood' AND e3.object_id = n.node_id
AND e1.edge_id <> e2.edge_id
AND e1.edge_id <> e3.edge_id
AND e2.edge_id <> e3.edge_id
ORDER BY hop1, hop2, hop1_uri, hop2_uri LIMIT 200;
```

## Memgraph (Cypher)

```
MATCH (a:Concept {name: 'blood'})-[]->(y:Concept)-[]->(z:Concept)-[]->(a
)
RETURN DISTINCT y.uri AS hop1_uri, y.name AS hop1, z.uri AS hop2_uri, z.
    name AS hop2
ORDER BY hop1, hop2, hop1_uri, hop2_uri LIMIT 200;
```

## 2.6 Neighborhood Degree Profile

**Task:** Given a concept, find all 1-hop neighbors in order of edge's degree.

## MySQL

```
SELECT nbr.uri AS uri, nbr.name AS name, COUNT(DISTINCT e2.edge_id) AS
    out_degree
FROM nodes n
JOIN edges e1 ON e1.subject_id = n.node_id
JOIN edges e2 ON e2.subject_id = e1.object_id
JOIN nodes nbr ON nbr.node_id = e1.object_id
WHERE n.name = 'horse'
GROUP BY nbr.uri, nbr.name
ORDER BY out_degree DESC, uri LIMIT 50;
```

## Memgraph (Cypher)

```
MATCH (:Concept {name: 'horse'})-[]->(nbr:Concept)-[r2]->(
)
RETURN nbr.uri AS uri, nbr.name AS name, count(DISTINCT r2) AS
    out_degree
ORDER BY out_degree DESC, uri LIMIT 50;
```

## 3 FAMILY C (Global Retrieval)

### 3.1 Concept Hubs Ranking

**Task:** Rank the top-10 concepts in terms of most outgoing edges.

#### MySQL

```
SELECT n.uri AS uri, n.name AS name, COUNT(*) AS out_degree
FROM edges e JOIN nodes n ON n.node_id = e.subject_id
GROUP BY n.uri, n.name
ORDER BY out_degree DESC, uri LIMIT 10;
```

#### Memgraph (Cypher)

```
MATCH (n:Concept)-[r]->(:Concept)
RETURN n.uri AS uri, n.name AS name, count(r) AS out_degree
ORDER BY out_degree DESC, uri LIMIT 10;
```

### 3.2 Relation-Class Type Matrix

**Task:** Group all the edges in the graph per (relation, class type) pair.

#### MySQL

```
SELECT r.relation_name AS relation, e.edge_class AS edge_class, COUNT(*)
      AS n_edges, AVG(e.weight) AS avg_w
FROM edges e JOIN relations r ON r.relation_id = e.relation_id
GROUP BY r.relation_name, e.edge_class
ORDER BY n_edges DESC, relation, edge_class;
```

#### Memgraph (Cypher)

```
MATCH (s:Concept)-[r]->(o:Concept)
RETURN type(r) AS relation,
      (CASE WHEN s:EntityNode THEN 'E' WHEN s:ActionEventNode THEN 'A'
            ELSE 'P' END)
      + '2' +
      (CASE WHEN o:EntityNode THEN 'E' WHEN o:ActionEventNode THEN 'A'
            ELSE 'P' END) AS edge_class,
      count(r) AS n_edges, avg(r.weight) AS avg_w
ORDER BY n_edges DESC, relation, edge_class;
```

## 4 FAMILY D (Schema and Integrity Findings)

### 4.1 Violation Check

**Task:** Count how many edges are violating the a priori type contract.

## MySQL

```
SELECT COUNT(*) AS violations
FROM v_edges
WHERE NOT permitted;
```

## Memgraph (Cypher)

```
MATCH (s:Concept)-[r]->(o:Concept)
MATCH (rt:Schema:RelationType {name: type(r)})
OPTIONAL MATCH (rt)-[:PERMITS]->(c:Schema:EdgeClass)
    WHERE c.name = (CASE WHEN s:EntityNode THEN 'E'
                        WHEN s:ActionEventNode THEN 'A' ELSE 'P' END) +
                    '2' +
                    (CASE WHEN o:EntityNode THEN 'E'
                        WHEN o:ActionEventNode THEN 'A' ELSE 'P' END)
WITH r, c WHERE c IS NULL
RETURN count(r) AS violations;
```

## 4.2 Invalid Insert

**Task:** Write a query that inserts an invalid relation with respect to the edge classes contract.

## MySQL

```
-- Prerequisite
SELECT node_id FROM nodes WHERE node_type = 'EntityNode'
ORDER BY node_id LIMIT 4;

-- (a) Truthful class, unpermitted combination: accepted, then visible
INSERT INTO edges (subject_id, subject_type, object_id, object_type,
relation_id, edge_class, weight)
VALUES (10, 'EntityNode', 11, 'EntityNode', 20, 'E2E', 1.0);

SELECT COUNT(*) FROM v_edges WHERE NOT permitted;

-- (b) Lying edge_class: ERROR 3819
INSERT INTO edges (subject_id, subject_type, object_id, object_type,
relation_id, edge_class, weight)
VALUES (10, 'EntityNode', 12, 'EntityNode', 20, 'E2P', 1.0);

-- (c) Lying endpoint type: ERROR 1452
INSERT INTO edges (subject_id, subject_type, object_id, object_type,
relation_id, edge_class, weight)
VALUES (10, 'ActionEventNode', 13, 'EntityNode', 20, 'A2E', 1.0);
```

#### Memgraph (Cypher)

```
CREATE (a:Concept:EntityNode {uri: '/c/en/_demo_a', name: '_demo_a',  
                                pos: 'n', label_source: 'uri'}),  
        (b:Concept:EntityNode {uri: '/c/en/_demo_b', name: '_demo_b',  
                                pos: 'n', label_source: 'uri'})  
CREATE (a)-[:HasProperty {weight: 1.0}]->(b);  
// accepted silently then D1 reports violations = 1: the query caught  
   what the engine did not.
```

### 4.3 Concept Delete

**Task:** Given a concept, delete it from the dataset.

#### MySQL

```
-- Setup  
INSERT INTO nodes (node_id, uri, name, pos, node_type, label_source)  
VALUES (292000, '/c/en/_bench_d3', '_bench_d3', 'n', 'EntityNode', 'uri')  
);  
  
INSERT INTO edges (subject_id, subject_type, object_id, object_type,  
                    relation_id, edge_class, weight) VALUES  
  (292000, 'EntityNode', 10, 'EntityNode', 33, 'E2E', 1.0),  
  (292000, 'EntityNode', 11, 'EntityNode', 33, 'E2E', 1.0),  
  (292000, 'EntityNode', 12, 'EntityNode', 33, 'E2E', 1.0);  
  
-- Step 1 - Wrong order (ERROR 1451)  
DELETE FROM nodes WHERE node_id = 292000;  
  
-- Step 2 - Correct order  
START TRANSACTION;  
DELETE FROM edges WHERE subject_id = 292000 OR object_id = 292000;  
DELETE FROM nodes WHERE node_id = 292000;  
COMMIT;
```

#### Memgraph (Cypher)

```
-- Setup  
CREATE (d:Concept:EntityNode {uri: '/c/en/_bench_d3', name: '_bench_d3',  
                                pos: 'n', label_source: 'uri'})  
  
WITH d  
MATCH (t:Concept:EntityNode)  
WITH d, t LIMIT 3  
CREATE (d)-[:RelatedTo {weight: 1.0}]->(t);  
  
-- Deletion step  
MATCH (n:Concept {uri: '/c/en/_bench_d3'})  
DETACH DELETE n;
```